Islamic University of Technology

Machine Learning Lab

CSE 4622

Lab 4: Neural Networks from Scratch

Author:
Ishmam Tashdeed
CSE, IUT

Contents

1 Introduction

In the previous lab you implemented Linear Regression but in this lab, we will tackle the problem of classification. The fundamental limitation of any linear model is that it can only learn a *linear* decision boundary. Real-world data, including clinical data like the heart disease dataset, often has non-linear patterns that a single weighted sum cannot capture.

A **Neural Network (NN)** overcomes this by stacking multiple layers of neurons, where each layer applies a linear transformation followed by a non-linear **activation function**. The result is a model that can learn arbitrarily complex decision boundaries given enough layers and neurons.

In this lab we build a two-layer neural network **entirely from scratch** — no frameworks, just Python lists and `math`. The architecture is:

$$\text{Input Layer} \xrightarrow{\mathbf{W}^{(1)},\mathbf{b}^{(1)}} \text{Hidden Layer (ReLU)} \xrightarrow{\mathbf{W}^{(2)},\mathbf{b}^{(2)}} \text{Output Neuron (Sigmoid)}$$

Data loading, preprocessing, and the overall structure of gradient descent were covered in previous labs. Starter code for those steps is provided. The focus here is on the new concepts: forward propagation, backpropagation, and weight initialization.

By the end of this lab you will be able to:

- Describe what happens during the forward pass of a neural network.
- Implement ReLU and sigmoid activation functions.
- Compute Binary Cross-Entropy loss for a neural network.
- Derive and implement backpropagation for a two-layer network using loops.
- Implement L1 and L2 regularization for neural networks.
- Compare loss and accuracy curves across unregularized, L1, and L2 variants.
- (Bonus) Compare your implementation against `sklearn`'s `MLPClassifier`.

Architecture Overview

Our network takes d input features and produces a single probability output. The hidden layer has H neurons (we will use $H = 16$).

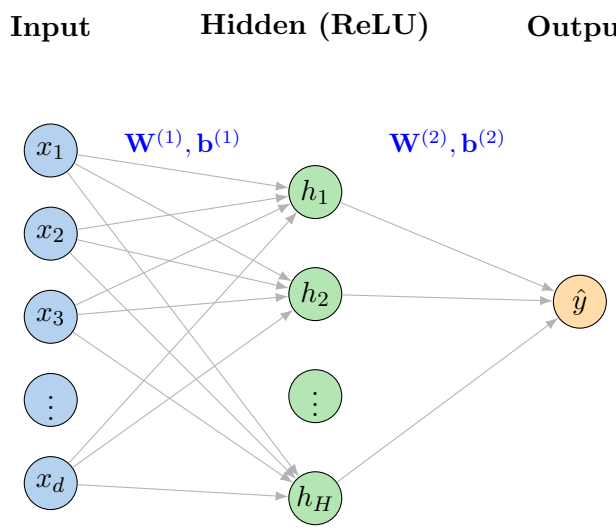

Figure 1: Two-layer neural network architecture used in this lab. The hidden layer uses ReLU activation; the output neuron uses Sigmoid.

The network has the following parameter shapes:

Parameter	Shape	Meaning
$\mathbf{W}^{(1)}$	$(H \times d)$	Weights from input to hidden layer
$\mathbf{b}^{(1)}$	$(H,)$	Biases of hidden neurons
$\mathbf{W}^{(2)}$	$(1 \times H)$	Weights from hidden to output
$\mathbf{b}^{(2)}$	$(1,)$	Bias of output neuron

2 Data Loading and Preprocessing

This section reuses the complete preprocessing pipeline from Lab 3. Run the starter code and verify the output. **No re-implementation is needed here.**

Expected Output

```
Training set shape:  (734, 15) | Testing set shape:  (184, 15)
```

3 Neural Network Theory

3.1 Activation Functions

An activation function introduces non-linearity after each linear transformation. Without activations, stacking multiple layers would collapse to a single linear layer and offer no expressive advantage.

3.1.1 ReLU (Hidden Layer)

The **Rectified Linear Unit** is the standard choice for hidden layers:

$$\mathrm{ReLU}(z) = \max(0,\, z) \tag{1}$$

Its derivative is simple: 1 when $z > 0$, and 0 otherwise. This is used during backpropagation.

3.1.2 Sigmoid (Output Layer)

The output neuron uses the sigmoid from Lab 4:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

The output $\hat{y} = \sigma(z) \in (0, 1)$ is interpreted as $P(\text{HeartDisease} = 1 \mid \mathbf{x})$.

3.2 Forward Propagation

For a single input sample $\mathbf{x} \in \mathbb{R}^d$, the forward pass computes:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \quad \in \mathbb{R}^H \tag{3}$$

$$\mathbf{a}^{(1)} = \mathrm{ReLU}\big(\mathbf{z}^{(1)}\big) \quad \in \mathbb{R}^H \tag{4}$$

$$z^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + b^{(2)} \quad \in \mathbb{R} \tag{5}$$

$$\hat{y} = \sigma\big(z^{(2)}\big) \quad \in (0, 1) \tag{6}$$

Note

$\mathbf{z}^{(1)}$ is called the **pre-activation** (input to the activation function) and $\mathbf{a}^{(1)}$ is the **post-activation** (output of the activation function). We must store $\mathbf{z}^{(1)}$ during the forward pass because we need it to compute the ReLU derivative in backpropagation.

3.3 Loss Function — Binary Cross-Entropy

We use the same Binary Cross-Entropy (BCE) loss as in Lab 4:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)}\log\hat{y}^{(i)} + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right] \tag{7}$$

3.4 Backpropagation

Backpropagation applies the **chain rule** to compute how the loss changes with respect to every parameter. We work *backwards* from the output layer to the input layer.

3.4.1 Output Layer Gradients

For each sample i, the error signal at the output is:

$$\delta_i^{(2)} = \hat{y}^{(i)} - y^{(i)} \tag{8}$$

(This is the same elegant result as in logistic regression — the BCE + sigmoid pairing cancels the sigmoid derivative.) The gradients are:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{W}^{(2)}} = \frac{1}{n}\sum_{i=1}^{n}\delta_i^{(2)}\cdot\mathbf{a}^{(1,i)\top}, \qquad \frac{\partial\mathcal{L}}{\partial b^{(2)}} = \frac{1}{n}\sum_{i=1}^{n}\delta_i^{(2)} \tag{9}$$

3.4.2 Hidden Layer Gradients

The error signal is *propagated back* through $\mathbf{W}^{(2)}$ and through the ReLU:

$$\boldsymbol{\delta}_i^{(1)} = \left(\mathbf{W}^{(2)\top}\delta_i^{(2)}\right)\odot\mathbf{1}\left[\mathbf{z}^{(1,i)} > 0\right] \tag{10}$$

where $\odot$ denotes element-wise multiplication and $\mathbf{1}[\mathbf{z}^{(1,i)} > 0]$ is the ReLU derivative (1 where pre-activation was positive, 0 elsewhere). The gradients are:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{\delta}_i^{(1)} \cdot \mathbf{x}^{(i)\top}, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{\delta}_i^{(1)} \tag{11}$$

3.4.3 Gradient Descent Update

Each parameter is updated by subtracting the learning-rate-scaled gradient:

$$\theta \leftarrow \theta - \alpha \cdot \frac{\partial \mathcal{L}}{\partial \theta} \quad \text{for each parameter } \theta \in \left\{ \mathbf{W}^{(1)},\, \mathbf{b}^{(1)},\, \mathbf{W}^{(2)},\, b^{(2)} \right\} \tag{12}$$

3.5 Regularization

For neural networks, the L1 and L2 penalties apply to **all weight matrices** (not biases):

$$\mathcal{L}_{\text{Ridge}} = \mathcal{L}_{\text{BCE}} + \lambda \left(\sum_{j,k} \left[W_{jk}^{(1)} \right]^2 + \sum_{k} \left[W_{k}^{(2)} \right]^2 \right) \tag{13}$$

$$\mathcal{L}_{\text{Lasso}} = \mathcal{L}_{\text{BCE}} + \lambda \left(\sum_{j,k} \left| W_{jk}^{(1)} \right| + \sum_{k} \left| W_{k}^{(2)} \right| \right) \tag{14}$$

The gradient additions are:

- L2: add $2\lambda W_{jk}$ to each weight gradient.
- L1: add $\lambda \cdot \text{sign}(W_{jk})$ to each weight gradient.

Bias gradients are **not** modified by regularization.

4 Additional Resources

- **Heart Failure Prediction Dataset:** https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction
- **Scikit-learn MLPClassifier:** https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
- **Backpropagation explained visually (3Blue1Brown):** https://www.youtube.com/watch?v=Ilg3gGewQ5U
- **CS231n Backpropagation Notes:** https://cs231n.github.io/optimization-2/
- **Andrew Ng's ML Course (Neural Networks):** https://www.coursera.org/learn/machine-learning

5 Tasks

Complete all tasks in your Colab notebook. Follow the given template when writing your code.

Task 1: Implement Missing Code Snippets

- Implement all the missing code snippets.
- Train the model for 300 epochs with a learning rate of 0.05 and 16 hidden layer neurons (H).

Task 2: Comparison and Analysis

- Retrain the model with $H \in \{4, 8, 16, 32\}$ and plot the final test accuracy vs. H on a line plot. Does a larger hidden layer always improve test accuracy?
- In a Markdown cell (3–5 sentences): What is the role of the hidden layer size H? What happens if H is too small? What happens if H is very large?
- Retrain the model with $\alpha \in \{0.001, 0.01, 0.05, 0.1\}$ and plot the final test accuracy vs. α on a line plot. What learning rate improves test accuracy?

Task 3: Custom Learning Rate Scheduler

- Implement a learning rate scheduler that decreases the learning rate by one-tenth every 100 epochs.
- Implement a learning rate scheduler that gradually increases and then decreases the learning rate during the training in a range of $\alpha \in [0.001, 0.1]$.
- Compare the two learning rate schedulers.

Task 4 (Bonus): Scikit-Learn Comparison

- Implement and train a neural network model using sklearn. Print the full `classification_report` for the best sklearn model.
- Compare test accuracy of your implementation vs. sklearn. Explain any difference in a Markdown cell.
- *Extra challenge:* Implement **mini-batch gradient descent** in your training loop. Instead of computing gradients over all training samples at once, randomly sample a mini-batch of 32 samples per iteration. Compare convergence speed (epochs to reach 85% test accuracy) against your full-batch implementation.

6 Submission Instructions

- Complete all tasks (Task 1–3, and optionally Task 4) in a single Google Colab notebook.
- Name your notebook: `ML_Lab4_YOUR_STUDENT_ID`
- Ensure all cells are executed and all outputs (including plots) are visible.
- Download the notebook as a **Python File** (`.py`) and submit via the course portal.